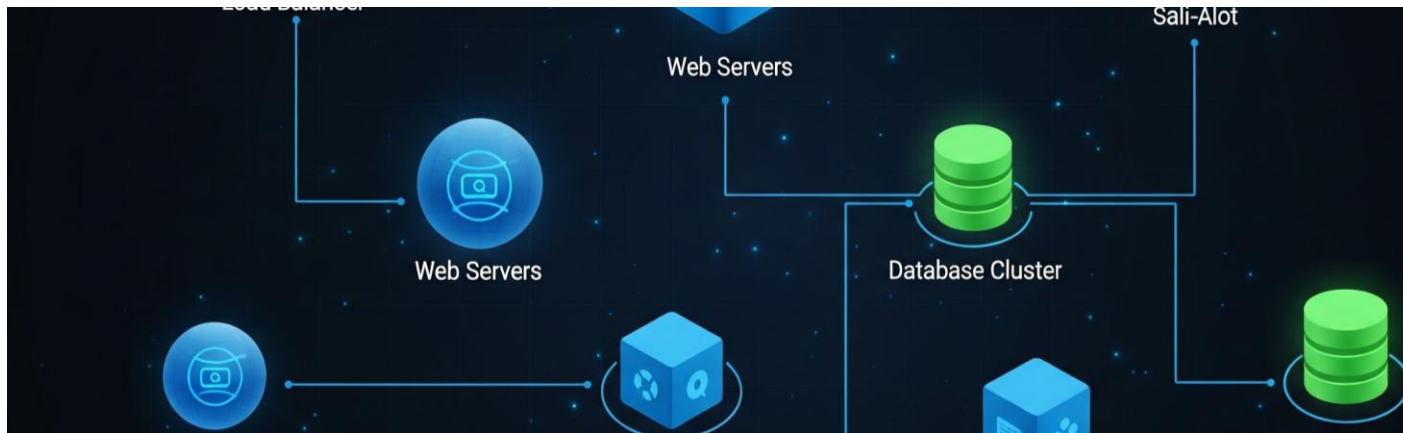




Product Requirements Document (PRD): Live Chat Rooms Feature

1. Executive Summary

This document outlines the requirements and architecture for the new **Live Chat Rooms** feature. This feature will provide users with real-time communication capabilities within custom-created chat rooms. The implementation is strictly **additive and modular**, ensuring zero impact on the existing stable backend, database schema (via new, separate tables only), and existing application logic. The design prioritizes scalability, extensibility, and robust security, built on a dedicated WebSocket architecture (Socket.IO) and decoupled services.

2. Goals

The primary goals of this project are:

- Successfully implement the Live Chat Rooms feature as a fully modular and isolated system.
- Enable authenticated users to create, discover, and join chat rooms.
- Establish a highly scalable, real-time communication channel compatible with horizontal scaling and high concurrency (10k+ users).
- Ensure the feature adheres to all non-breaking change constraints and security standards.
- Provide a foundation for future features, such as user banning, moderation, and private room access control.

3. Non-Goals

The following are explicitly **out of scope** for the initial release:

- Modification of any existing database tables or features (this is an add-only feature).
- Development or alteration of the existing UI (focus is on backend logic, APIs, and WebSockets).
- Implementation of advanced moderation tools (e.g., kicking, banning, message deletion by mods).
- File sharing or media embedding within chat messages.
- Support for message editing or deletion by users.
- Push notifications for new messages or room activities.

4. User Stories

ID	User Story	Priority
US-1	As an authenticated user, I can create a new chat room by providing a name, description, purpose, maximum member limit, rules, and visibility.	High
US-2	As a user, I can view a dynamically updated list of available Public chat rooms, including the room name, short description, and current member count.	High
US-3	As a user, I can search the list of available rooms by room name.	Medium
US-4	As a user, when I attempt to join a room, I must be shown the full rules and explicitly agree to them before being granted access.	High
US-5	As a member of a room, I can send real-time text messages to all other members in that room.	High

ID	User Story	Priority
US-6	As a member of a room, I can receive real-time messages sent by other members of the room.	High
US-7	As a member of a room, I can see a real-time list of all currently joined members.	High
US-8	As a user, I can leave a room and be removed from the real-time communication channel.	High
US-9	As a room creator, I can delete the room, which removes it from the listing and disconnects all active members.	High

5. Functional Requirements

FR 1.0 Room Creation Flow

ID	Requirement
FR-1.1	An authenticated user must be able to create a room via a dedicated API endpoint.
FR-1.2	The room creation request must include and store: Room Name, Description, Purpose, Category (optional), Visibility (Public/Private), Max Members Limit, Room Rules, and Optional Joining Requirements.
FR-1.3	The system must record the <code>user_id</code> of the room creator.
FR-1.4	The system must store an audit log of the creator's "Creator Agreement Confirmation" (boolean) within the room metadata.

ID	Requirement
FR-1.5	Upon successful creation, the room must immediately be available in the public room listing (if Public).

FR 2.0 Room Listing & Search

ID	Requirement
FR-2.1	A dedicated API endpoint must provide a paginated list of active Public rooms.
FR-2.2	The listing data must include: Room Name, Short Description, Current Member Count, and a Join Button indicator.
FR-2.3	The listing must be dynamically updated in near real-time (e.g., via poll or server-sent events for member count changes).
FR-2.4	The listing API must support filtering/searching rooms by Room Name.

FR 3.0 Room Joining Flow

ID	Requirement
FR-3.1	Upon initial "Join" click, the user must be presented with the Room Description, Purpose, Rules, and Joining Requirements (retrieved via an API).
FR-3.2	The user must explicitly tick an "I Agree to Follow Rules" checkbox.
FR-3.3	A dedicated API endpoint must handle the final join request, accepting the user's explicit rule acceptance confirmation.

ID	Requirement
FR-3.4	The user can only be added to the room if they confirm agreement and the room has not reached the Max Members Limit.
FR-3.5	The system must store a record in the <code>room_rule_acceptance_log</code> table upon successful room joining for audit purposes.
FR-3.6	Users who have already joined a room must be prevented from re-joining (duplicate join prevention).

FR 4.0 Real-Time Communication

ID	Requirement
FR-4.1	Upon successful join, the user's client must establish a WebSocket connection and join a specific Socket.IO room (namespaced to the chat feature).
FR-4.2	The server must support broadcasting messages from one user to all other users in the same socket room.
FR-4.3	The server must persist all sent messages to the <code>chat_messages</code> table before broadcasting.
FR-4.4	The server must maintain and broadcast the current list of connected members within a socket room upon join/leave events.
FR-4.5	When a user leaves (explicitly or disconnects), they must be removed from the socket room and their membership status updated in the database.

6. Non-Functional Requirements

Scalability (NFR-S)

ID	Requirement
NFR-S-1	The system must be designed to handle 10,000+ concurrent users actively messaging across all rooms.
NFR-S-2	The WebSocket server architecture must support horizontal scaling using a distributed messaging mechanism (e.g., Socket.IO Redis Adapter).
NFR-S-3	The architecture must be load-balancer friendly, ensuring sticky sessions are either not required or correctly managed (if required by Socket.IO).
NFR-S-4	Chat history must be retrieved from the database, not stored in memory, to conserve server resources.

Performance (NFR-P)

ID	Requirement
NFR-P-1	Message latency (send to receive) must be less than 200ms under typical load.
NFR-P-2	API response times for room creation/joining should be under 500ms (p95).

Maintainability & Extensibility (NFR-M)

ID	Requirement
NFR-M-1	All new components (controllers, services, routes, models) must be modular and

ID	Requirement
	isolated within a dedicated chat module or namespace.
NFR-M-2	The system must be extensible to support future features like user banning, room moderation, and private room invitation systems.
NFR-M-3	Strict adherence to the "ADD-ONLY" constraint is mandatory; no modifications to existing core logic.

Reliability (NFR-R)

ID	Requirement
NFR-R-1	The WebSocket layer must implement proper error handling and clean up for unexpected disconnections.
NFR-R-2	Database transactions must ensure atomicity for critical operations like room creation and member joining.
NFR-R-3	The system must implement "soft delete" logic for rooms and messages.

7. System Architecture Overview

The Live Chat Rooms feature will be implemented as a separate, decoupled micro-service or a self-contained module within the existing backend, utilizing its own dedicated database models and business logic.

Architectural Components

1. **Chat API Module:** Houses all RESTful endpoints for Room Creation, Listing, Joining, and Data Retrieval.
 - **Layering:** Dedicated Controllers (routes), Service Layer (business logic), and Repository/Model layer (database interaction).
2. **WebSocket Server:** A separate component or module managing the real-time communication.

- **Technology:** Socket.IO, operating on a dedicated namespace (e.g., `/chat`).
 - **Scaling:** Uses the Socket.IO Redis Adapter for pub/sub capabilities, enabling multiple WebSocket server instances to share state and broadcast across the cluster.
3. **Database (Additive Only):** Dedicated new tables (`chat_rooms`, `chat_room_members`, `chat_messages`, `room_rule_acceptance_log`).

8. Database Schema Proposal (Additive Only)

The following tables are proposed to be added. All tables must include: `id` (Primary Key), `created_at`, `updated_at`, and `is_deleted` (Soft Delete).

`chat_rooms`

Column	Data Type	Constraints
<code>id</code>	UUID	PK, NOT NULL
<code>creator_user_id</code>	UUID	FK to existing <code>users</code> table, NOT NULL
<code>name</code>	VARCHAR(100)	NOT NULL, UNIQUE
<code>description</code>	TEXT	NOT NULL
<code>purpose</code>	VARCHAR(255)	NOT NULL
<code>category</code>	VARCHAR(50)	NULL
<code>visibility</code>	ENUM	NOT NULL (Public, Private)
<code>max_members</code>	INTEGER	NOT NULL
<code>rules</code>	TEXT	NOT NULL
<code>joining_requirements</code>	TEXT	NULL

Column	Data Type	Constraints
creator_agreed_terms	BOOLEAN	NOT NULL
status	ENUM	NOT NULL (Active, Deleted, Suspended)

chat_room_members

Column	Data Type	Constraints
id	UUID	PK, NOT NULL
room_id	UUID	FK to <code>chat_rooms</code> , NOT NULL
user_id	UUID	FK to existing <code>users</code> table, NOT NULL
joined_at	TIMESTAMP	NOT NULL
is_active	BOOLEAN	NOT NULL (default TRUE)
is_banned	BOOLEAN	NOT NULL (default FALSE)
UNIQUE KEY	(room_id, user_id)	Ensures no duplicate joins

chat_messages

Column	Data Type	Constraints
id	UUID	PK, NOT NULL
room_id	UUID	FK to <code>chat_rooms</code> , NOT NULL

Column	Data Type	Constraints
sender_user_id	UUID	FK to existing <code>users</code> table, NOT NULL
content	TEXT	NOT NULL
sent_at	TIMESTAMP	NOT NULL
status	ENUM	NOT NULL (Sent, Failed)

`room_rule_acceptance_log`

Column	Data Type	Constraints
id	UUID	PK, NOT NULL
room_id	UUID	FK to <code>chat_rooms</code> , NOT NULL
user_id	UUID	FK to existing <code>users</code> table, NOT NULL
accepted_at	TIMESTAMP	NOT NULL
is_agreed	BOOLEAN	NOT NULL (must be TRUE)

9. API Design

All endpoints must be prefixed with a new, dedicated namespace, e.g., `/api/v1/chat`. Authentication is mandatory via existing middleware.

Method	Endpoint	Description
POST	<code>/api/v1/chat/rooms</code>	Create a new chat room.

Method	Endpoint	Description
GET	<code>/api/v1/chat/rooms</code>	Get paginated list of public rooms (with search/filter).
GET	<code>/api/v1/chat/rooms/{roomId}</code>	Get detailed room information (rules, description) for pre-join view.
POST	<code>/api/v1/chat/rooms/{roomId}/join</code>	Final step of room joining, including rule acceptance confirmation.
POST	<code>/api/v1/chat/rooms/{roomId}/leave</code>	Explicitly leave a room.
DELETE	<code>/api/v1/chat/rooms/{roomId}</code>	Delete a room (Creator only).
GET	<code>/api/v1/chat/rooms/{roomId}/messages</code>	Get paginated chat history for a room.

10. Socket Event Flow

The Socket.IO server will use a dedicated namespace: `/chat`.

Client to Server Events (Emit)

Event Name	Payload	Description
<code>joinRoom</code>	<code>{ roomId: string, token: string }</code>	Requests to join the real-time socket room after successful API join.
<code>sendMessage</code>	<code>{ roomId: string, content: string }</code>	Sends a new message to the room.
<code>leaveRoom</code>	<code>{ roomId: string }</code>	Explicitly leaves the socket room.

Server to Client Events (Listen)

Event Name	Payload	Description
message	<code>{ senderId: string, content: string, timestamp: string }</code>	Broadcasted to all users in the room when a new message is sent.
memberJoined	<code>{ userId: string, count: number }</code>	Broadcasted when a new user connects to the socket room.
memberLeft	<code>{ userId: string, count: number }</code>	Broadcasted when a user disconnects or explicitly leaves.
roomDeleted	<code>{ roomId: string }</code>	Sent to all users in the room when the room creator deletes it.

11. Security Considerations

Requirement	Implementation Strategy
Authentication	All room creation, joining, sending, and leaving operations must pass through existing authentication middleware. Socket connection requests must be authenticated via a token/session ID.
Input Validation	All API inputs (Room Name, Description, Content, etc.) must be strictly validated for type, length, and format.
Prevent Duplicate Joins	The <code>chat_room_members</code> table unique constraint (<code>room_id</code> , <code>user_id</code>) prevents duplicate membership records.
Rate Limit	Implement a server-side rate limiting mechanism on the <code>sendMessage</code> socket event to prevent DoS attacks and spamming. (e.g., 5 messages per 3 seconds per user).

Requirement	Implementation Strategy
Message Sanitization	All incoming chat message content must be sanitized (e.g., stripping HTML/script tags) before persistence and broadcasting to prevent XSS attacks.
Creator Protection	Logic to delete a room must verify the requesting user is the creator. Ownership transfer logic (future) is required before a creator can be removed/leave permanently.

12. Risk Assessment

Risk	Severity	Mitigation Strategy
Memory Leaks in Socket Layer	High	Implement rigorous clean-up logic for all socket events (<code>on('disconnect')</code> , <code>on('leaveRoom')</code>) to ensure proper garbage collection and prevent orphaned event listeners.
Database Lock Contention	Medium	Use optimized queries, proper indexing (<code>room_id</code> , <code>user_id</code>), and minimize writes during high-frequency operations (e.g., only persist messages, not member presence).
Breaking Existing System	High	Strict adherence to modular, additive design. Comprehensive unit and integration tests covering the new module in isolation.
Scalability Bottleneck	Medium	Proactive implementation of Socket.IO Redis Adapter from day one. Load testing to validate 10k+ concurrent user

Risk	Severity	Mitigation Strategy
		support before production release.

13. Rollout Plan

- Phase 1: Local Development & Unit Testing**
 - Develop API and WebSocket services in isolation.
 - Create all new database tables and models.
 - Complete unit tests for all new functions/services.
- Phase 2: Integration Testing (Staging)**
 - Deploy the new Chat Module to the staging environment.
 - Perform end-to-end integration tests with the UI (simulated client connections).
 - Load testing for 1,000 concurrent users.
- Phase 3: Beta Release (Controlled Group)**
 - Enable the feature for a small internal beta group (e.g., 50 power users).
 - Monitor performance, stability, and error logs closely for 1 week.
- Phase 4: General Availability**
 - Full production rollout.
 - Monitor key metrics: latency, error rates, and resource utilization.

14. Testing Strategy

The testing strategy will focus heavily on validating the non-interference constraint and the stability of the real-time layer.

Test Areas

Test Type	Focus Area	Description
Unit Tests	API validation, Service logic, Database transactions	Verify all functions work as expected (e.g., Room creation stores all metadata correctly).
Integration Tests	API-Database flow, Socket-API flow	Verify data flows correctly between the API, database, and WebSocket layers (e.g., Joining API call successfully

Test Type	Focus Area	Description
		adds user and triggers Socket <code>joinRoom</code> flow).
Non-Interference Tests	Existing features	Run regression tests on a set of existing features to confirm zero impact from the new module deployment.
Stress/Load Tests	WebSocket server and API	Simulate 10k concurrent users sending/receiving messages to test NFR-S-1 and NFR-P-1. Use tools like <code>k6</code> or <code>Gatling</code> .
Security Tests	Rate limiting, Input Sanitization, Auth	Test for injection attempts and ensure rate limits function correctly under load.